



Curso IMPY Tech

Aula 09

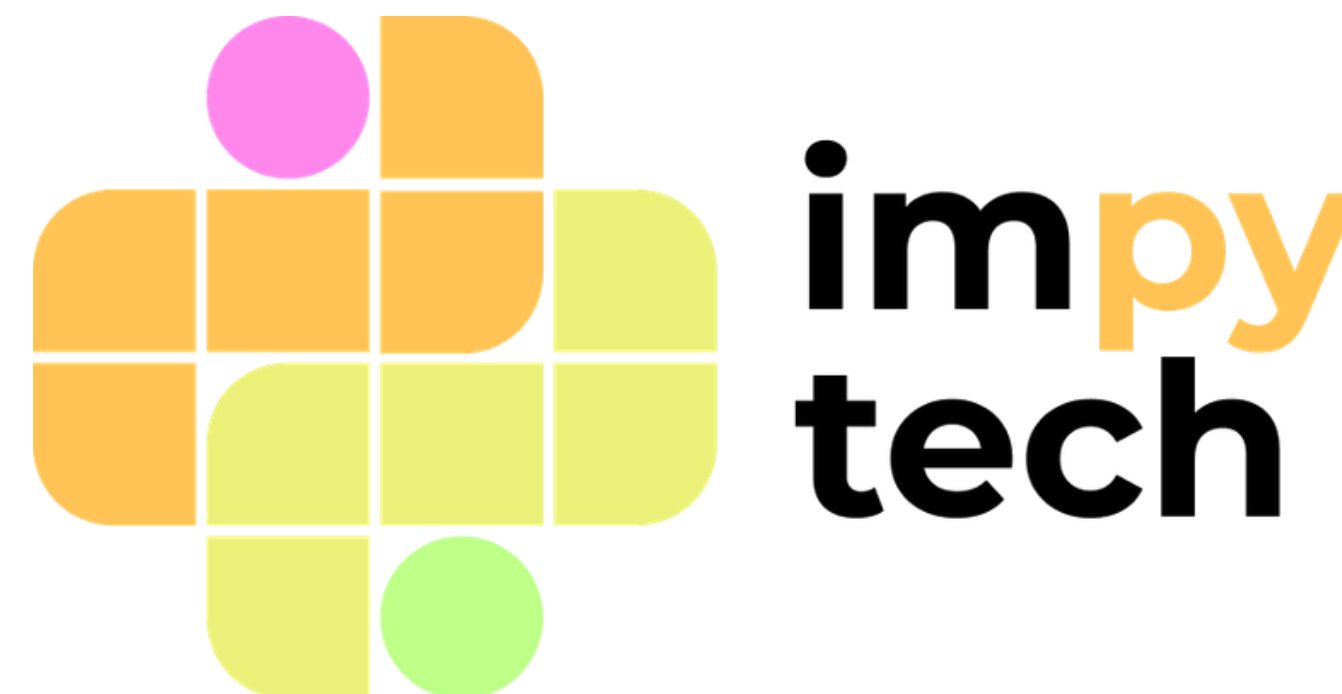
**POO, Incorporação de uso de LLM's e
Introdução ao Aprendizado de Máquina**

Pedro Alberti

Bruno Pereira

Bianca Zavadisk

Cristiane Sampaio



Imagine que você queira criar uma lista de pacientes em Python. Não existe algum tipo de dados básico que represente exatamente a informação que você quer: um paciente possui nome, idade, peso, altura, diagnósticos anteriores...

Uma estratégia é criar uma lista para cada “componente” de informação (uma lista de nomes, outra de alturas, etc). Funciona, mas isso deixa informações que se referem ao mesmo paciente em partes completamente diferentes do código e da memória do seu computador.

Outra possível estratégia é utilizar um dicionário para representar um paciente, em que cada componente é uma entrada do dicionário. Essa estratégia é muito boa em certas situações, mas a natureza fundamentalmente dinâmica de dicionários pode prejudicar eficiência e clareza do código.

Nessa aula, introduziremos uma terceira estratégia: se o nosso problema é que não existe algum tipo “Paciente”, por que não inventar esse tipo nós mesmos? Essa é a base do conceito de “classes”.

Classes e objetos

- Uma classe é uma declaração de tipo, um “modelo”.
- Uma variável cujo tipo é uma classe é chamada de objeto, ou instância da classe.

Exemplo 1

```
class Paciente():  
    def __init__(self, nome, idade): # Esse método roda toda vez que criamos um objeto  
        self.nome = nome  
        self.idade = idade  
  
Joao = Paciente("João", 23) # João é uma instância da classe Paciente  
print(Joao.nome) # Imprime "João"  
print(Joao.idade) # Imprime "23"
```

Exemplo 2

```
class Paciente():
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def ver_dados(self):
        print(
            self.nome + ": " \
            + f"{self.idade} anos"
        )

Joao = Paciente("João", 23)
print(Joao.nome)
print(Joao.idade)
Joao.ver_dados()
```

Classes e objetos

- Uma classe pode possuir funções associadas. Quando o primeiro parâmetro de uma função for “self”, chamamos essa função de método.
- “self” é um parâmetro que sempre se refere ao objeto no qual o método é chamado.
- Classes também podem ter elementos associados: por exemplo, “Joao.nome” é um elemento de tipo string associado à variável “Joao” de tipo Paciente. Chamamos esses elementos associados de atributos.
- Acessamos atributos de um objeto e chamamos métodos nesse objeto com a mesma sintaxe: “<objeto>.<atributo/método>”.

Exemplo 2

```
class Paciente():
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def ver_dados(self):
        print(
            self.nome + ": " \
            + f"{self.idade} anos"
        )

Joao = Paciente("João", 23)
print(Joao.nome)
print(Joao.idade)
Joao.ver_dados()
```

Classes e objetos

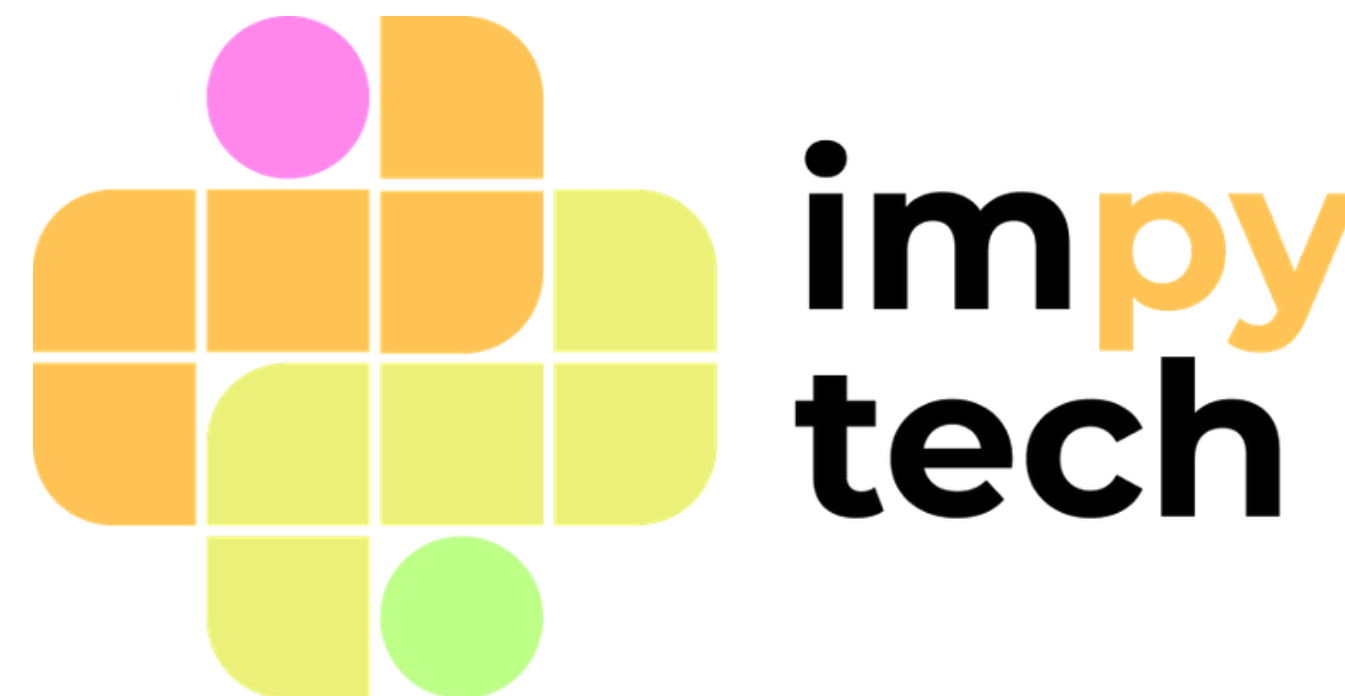
- Criamos uma instância de Paciente com uma sintaxe parecida com chamar uma função.
- Os argumentos usados na criação de uma instância são passados ao método mágico “__init__”, que guarda a lógica de inicialização da nossa classe.

Assim, classes servem para armazenar dados e lógica relacionados em um mesmo lugar, por meio de atributos e métodos, respectivamente.

Dessa forma, podemos criar modelos mais adequados para os problemas que estamos analisando do que os tipos básicos do Python, o que deixa nosso código mais limpo e elegante.



Métodos Mágicos



Métodos mágicos são métodos que informam ao Python capacidades adicionais da sua classe. Já vimos um deles: “__init__” guarda a lógica de inicialização de uma classe, rodando toda vez que criamos um objeto.

Métodos mágicos cobrem muitas funções: definir adição, subtração, multiplicação e outras operações aritméticas; definir indexação ou tamanho de uma lista; definir conversão em strings, etc.

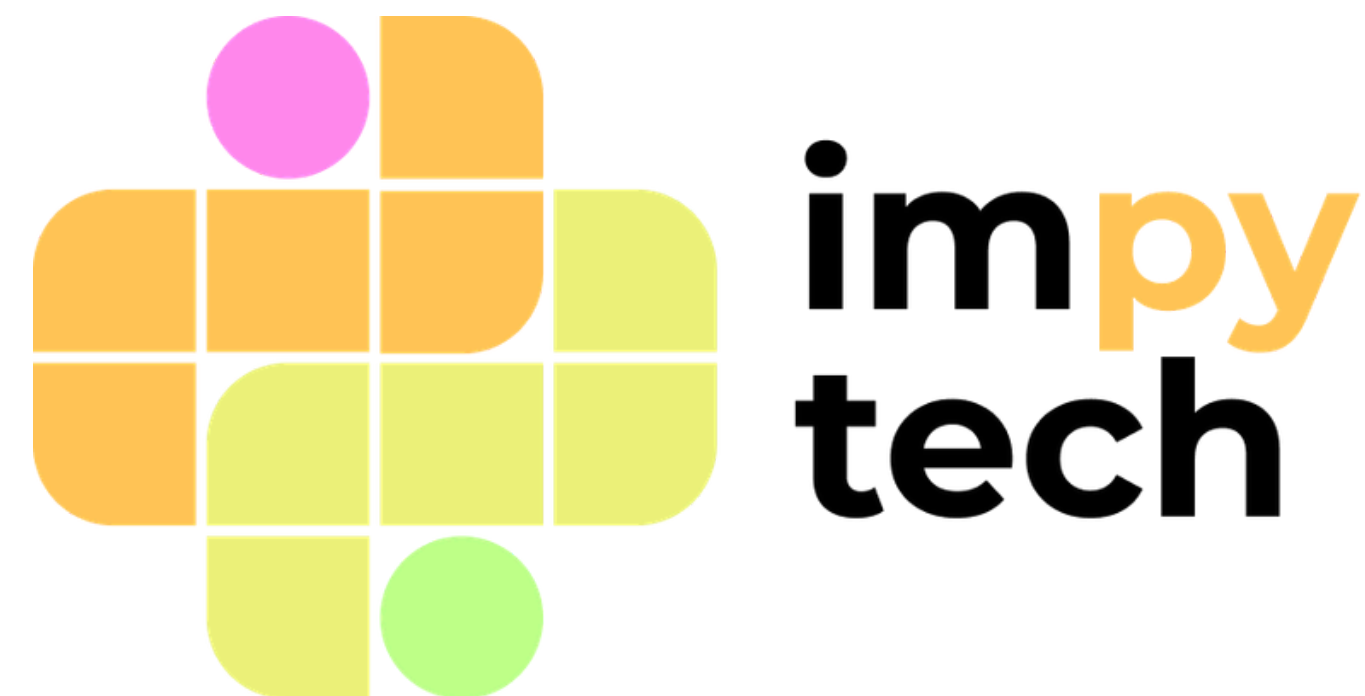
Em geral, cada método mágico está associado a uma sintaxe ou função especial embutida no Python.

Método	Descrição	Sintaxe
<code>__init__</code>	Cria uma instância da classe.	<code>Maria = Paciente()</code>
<code>__str__</code>	Representa um objeto como uma string.	<code>print(Maria)</code>
<code>__len__</code>	Retorna o tamanho de uma coleção de objetos.	<code>len(lista)</code>
<code>__getitem__</code>	Retorna o elemento no índice "x" de uma coleção de objetos.	<code>lista[x]</code>

Note que `__getitem__` não precisa necessariamente indexar por números! Por exemplo, se sua classe representa um banco de dados de informações de pacientes, pode ser mais conveniente indexar pelo nome ou pelo CPF do que pela posição numérica no banco de dados. O Python te dá flexibilidade para indexar do jeito que achar melhor.



Encapsulamento



Outra parte fundamental do paradigma da Programação Orientada a Objetos é o encapsulamento. Essencialmente, consiste em “proteger” a lógica interna de uma classe e só expor métodos seguros para interagir com ela.

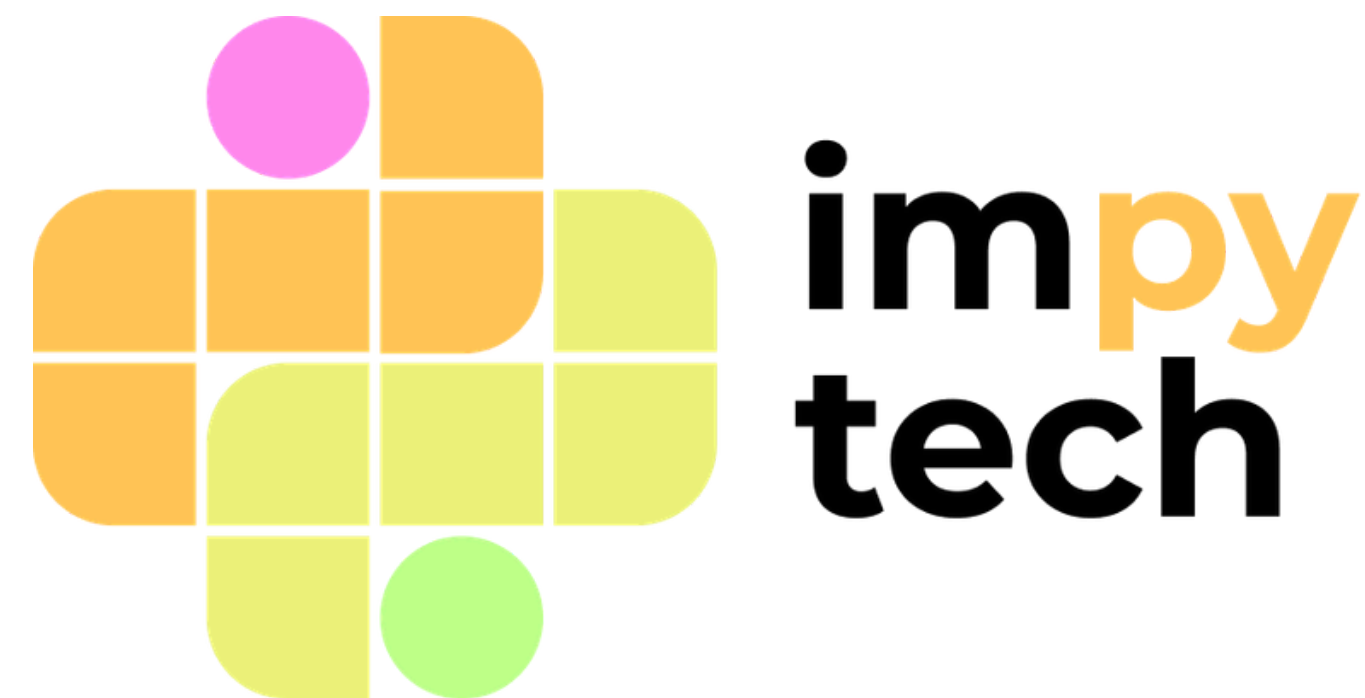
Usando nossa classe “Paciente” como exemplo, poderíamos criar um paciente com idade negativa, o que não faz sentido. Para garantir que a idade de um paciente é sempre positiva, poderíamos criar funções “get_idade” e “set_idade” que realizam a verificação da idade.

No entanto, o Python nos oferece uma alternativa melhor, por meio de decoradores. O decorador “@property” nos permite declarar um método como “acessor”, ou “getter”, de um atributo. Já o decorador “@<atributo>.setter” nos permite declara um método como “modificador”, ou “setter”, desse atributo. Em combinação, esses decoradores nos permitem criar “atributos inteligentes”, que executam código quando são lidos ou modificados.

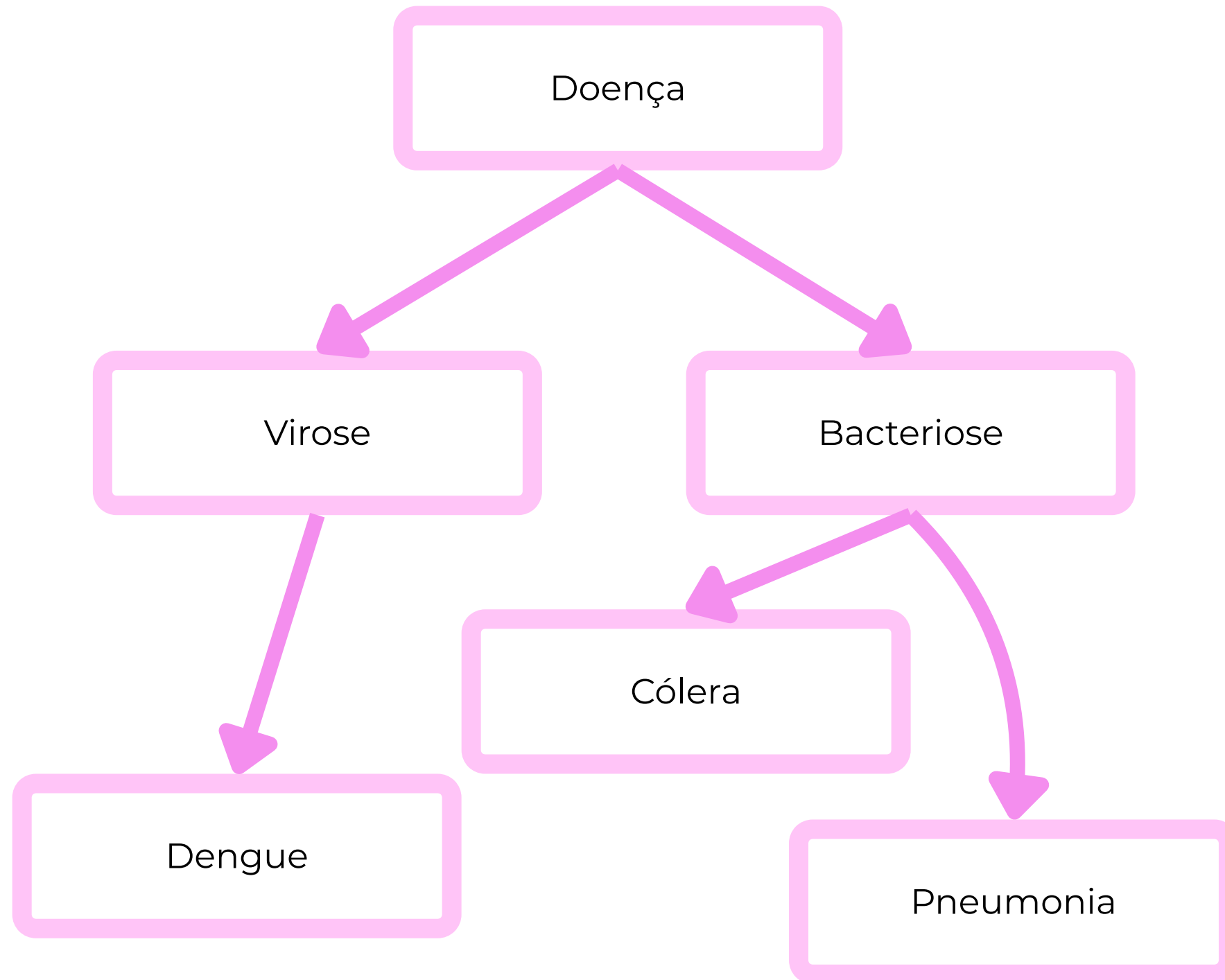
Por convenção, métodos e atributos cujo nome começam com um underscore (“_”) são tratados como “privados” e não devem ser acessados fora do código da classe em si.



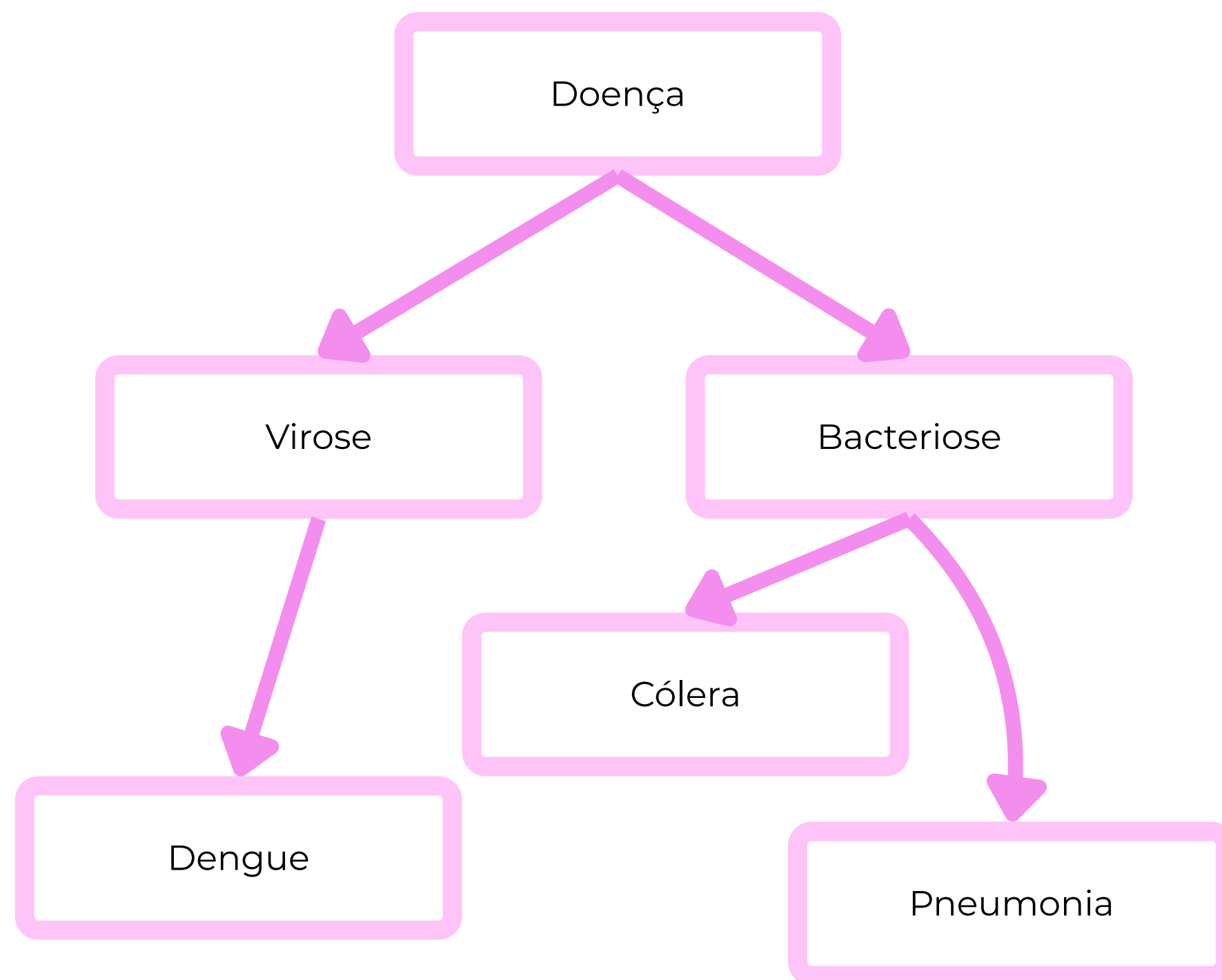
Herança



Muitas vezes, queremos representar um modelo que “estende” outro, ou seja, que possui todas as propriedades de algum outro modelo e mais algumas outras. Uma implementação tola disso seria criar duas classes e repetir todo o código da primeira na segunda. Como de costume, o Python nos oferece uma alternativa melhor: uma classe pode possuir uma “classe mãe” e “herar” todas as suas propriedades dela.



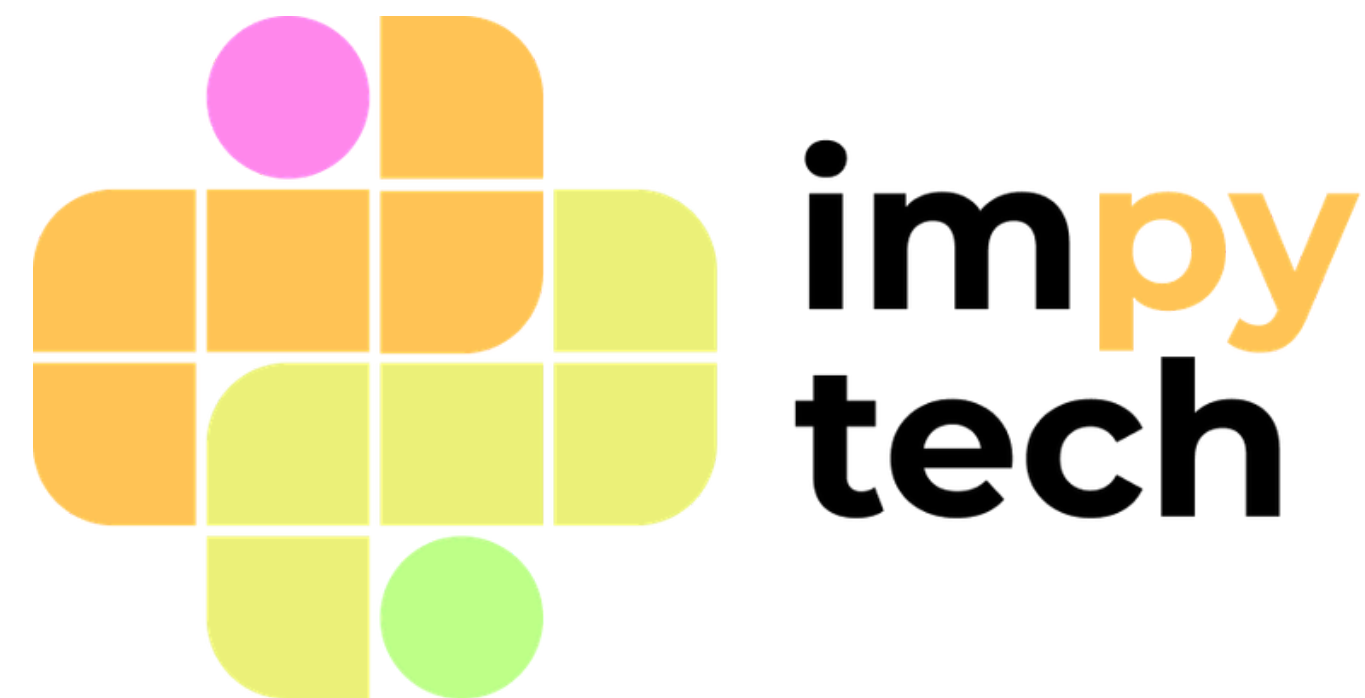
Ao lado, temos um exemplo de um “diagrama de classes”, onde cada classe aponta para suas classes filhas (aquelas que herdam suas propriedades). Nesse exemplo, uma classe ancestral “Doença” possui subclasses referentes aos tipos de doença, e cada uma delas possui subclasses referentes a doenças específicas.



A classe Doença define métodos e atributos referentes a doenças em geral, e seus descendentes definem métodos cada vez mais específicos. Por exemplo, a classe Bacteriose pode definir um método “antibióticos_recomendados”, que não faria sentido na classe virose, e cada subclasse de Bacteriose pode re-implementar esse método de forma mais específica.



Incorporação de uso responsável de LLM's



01

Exemplos gerais de usos ruins de IAs no dia a dia.

- 1.1 Explicar as falhas de segurança que podem acontecer por conta do mau uso de IA's.

02

Introdução à Engenharia de Prompt

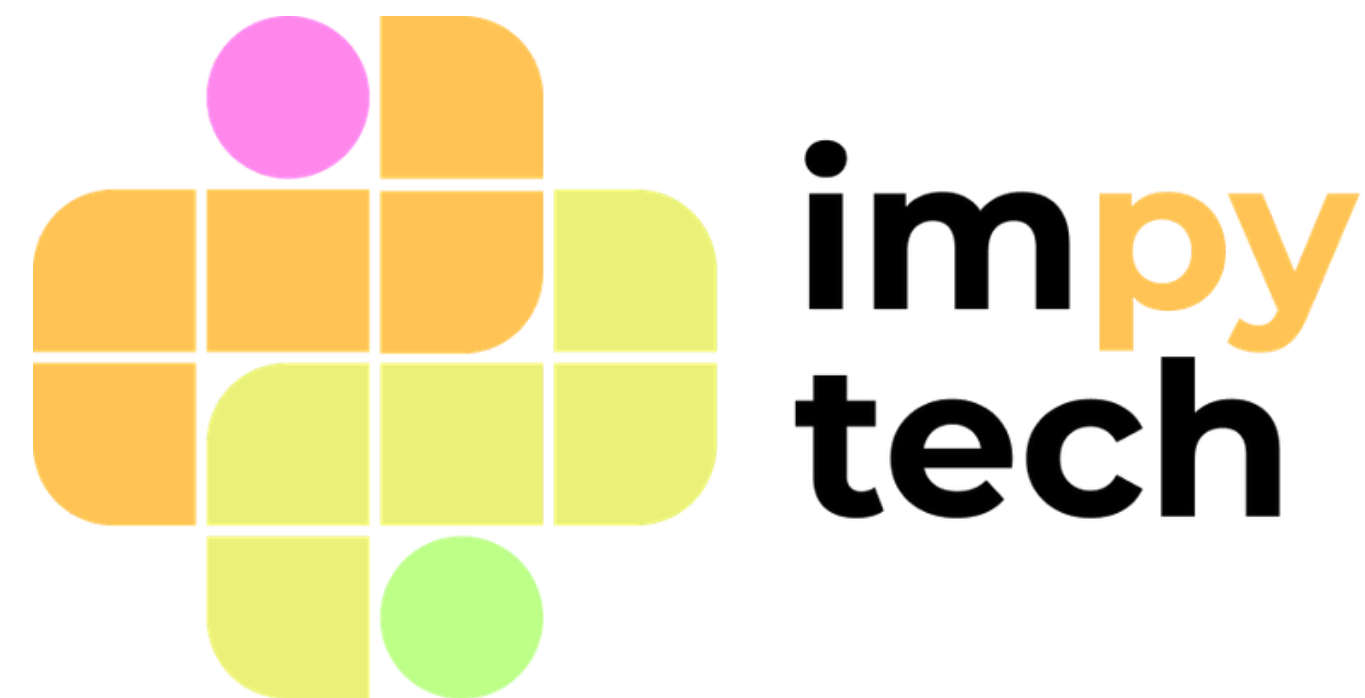
- 2.1 Exemplificar como podemos incorporar o uso de IA's em projetos, sem perder o controle do mesmo.

03

Reflexão sobre o que pode ser aprendido à nível de código quando se utiliza IA's.



Tema 1



- Na área da medicina, os problemas que aparecem muitas vezes envolvem dados sensíveis e importantes para as pessoas que as detêm.

- Na área da medicina, os problemas que aparecem muitas vezes envolvem dados sensíveis e importantes para as pessoas que as detêm.
- **Com a crescente das ferramentas de IA como ChatGPT, Claude e Gemini, muitas pessoas começaram a usar e acreditar cegamente sem nenhum tipo de revisão. Algo que não é certo.**

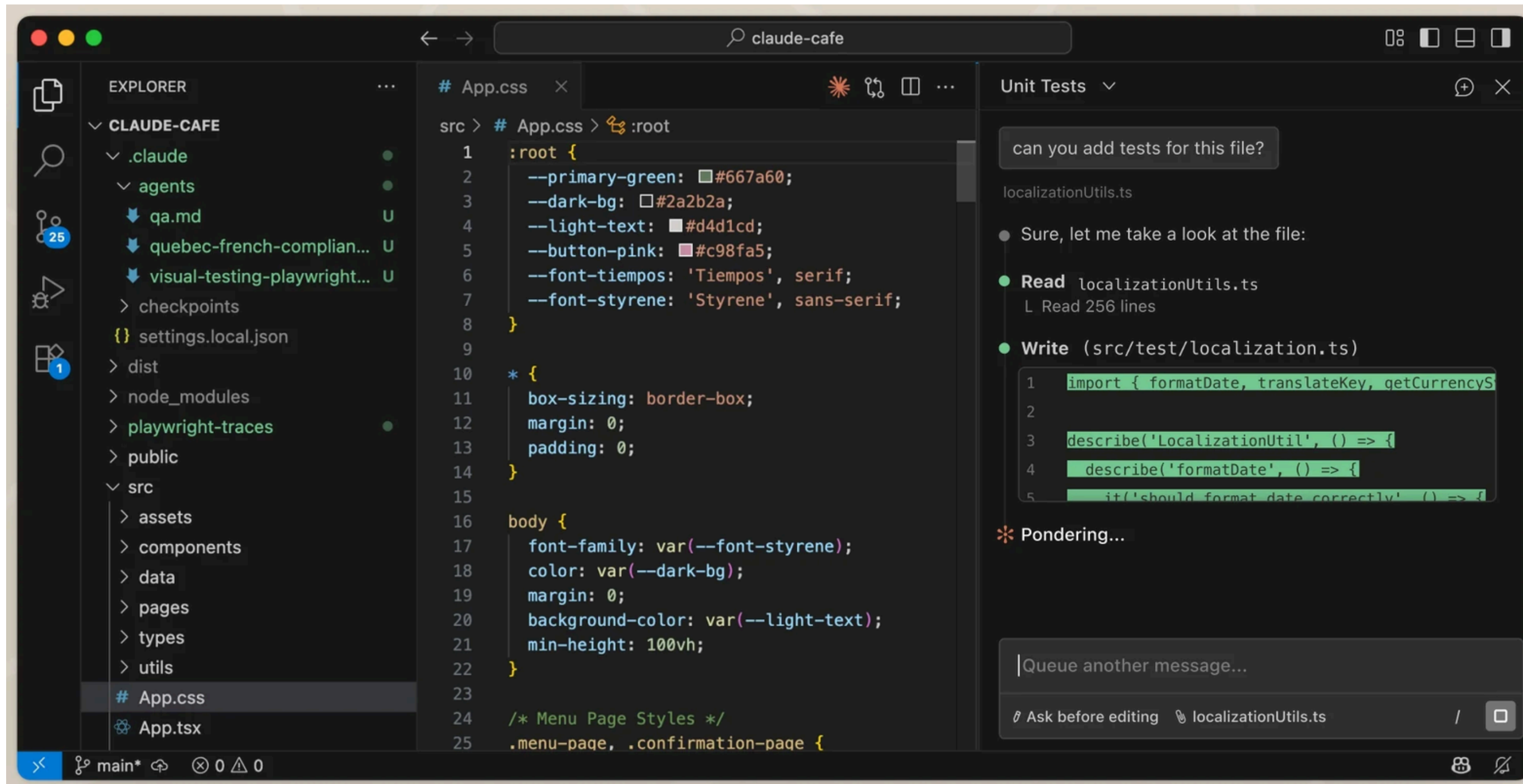


Figura 9.1: Ferramenta Claude Code integrada ao VSCode, facilitando o uso da LLM no manuseio de arquivos.

 TecMundo

Anthropic leva Claude Cowork para celular e navegador; saiba como funciona

O Claude Cowork enfim será integrado à versão web e ao app mobile do Claude, anunciou a Anthropic nesta terça-feira (7).

3 dias atrás



 Mitrade

A OpenAI adiciona o Codex ao aplicativo móvel ChatGPT à medida que a rivalidade com o Claude Code se intensifica

Codex mobile: Uma resposta competitiva ao Claude Code? No mês passado, a OpenAI deu ao Codex a capacidade de executar tarefas em segundo plano...

14 de mai. de 2026

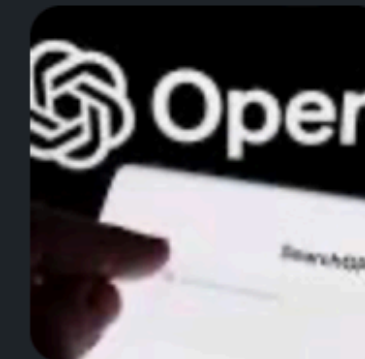
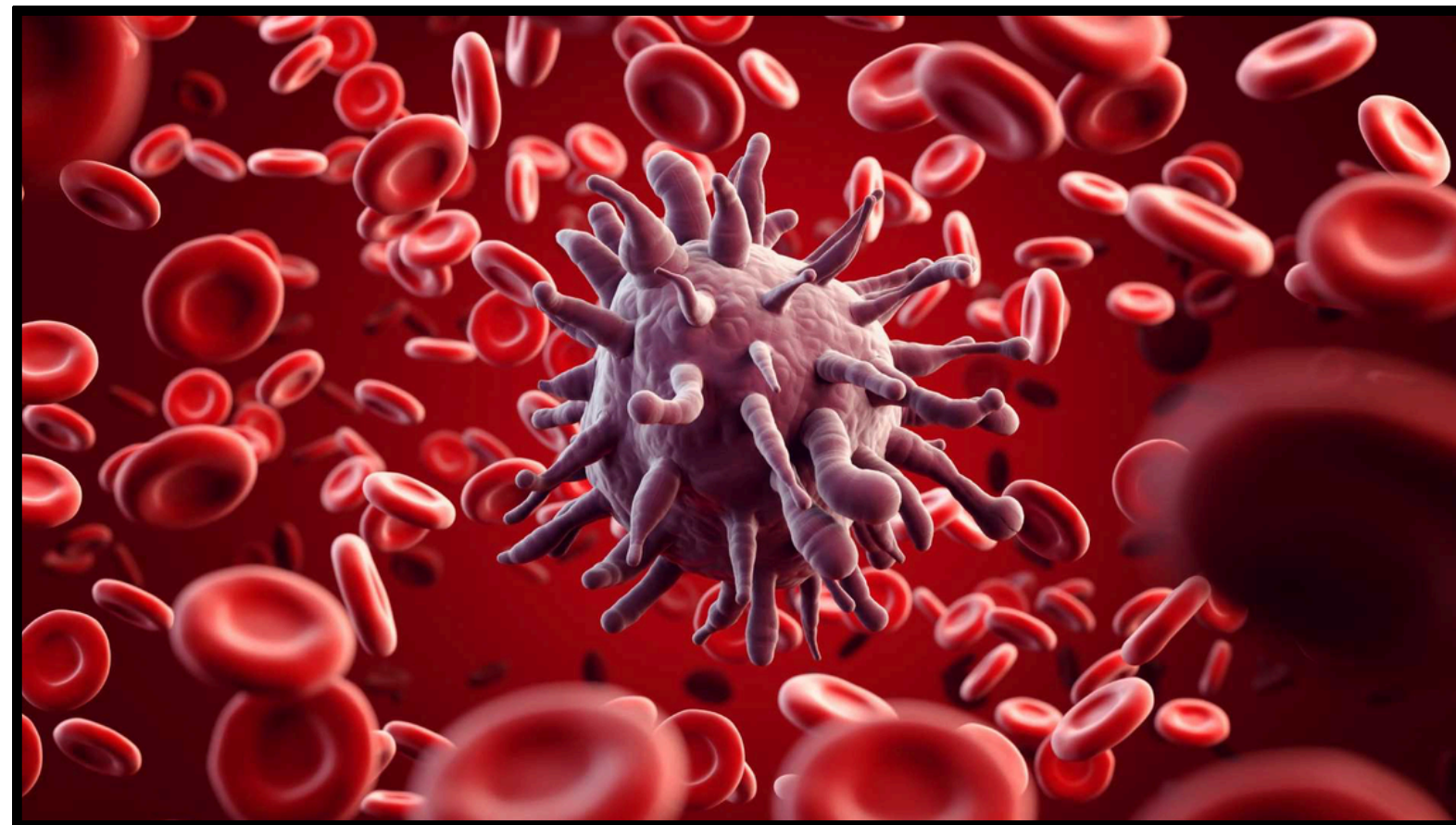


Figura 9.2: Ferramentas de IA generativa para código também estão sendo integradas aos navegadores e celulares.

- Na área da medicina, os problemas que aparecem muitas vezes envolvem dados sensíveis e importantes para as pessoas que as detêm.
- Com a crescente das ferramentas de IA como ChatGPT, Claude e Gemini, muitas pessoas começaram a usar e acreditar cegamente sem nenhum tipo de revisão. Algo que não é certo.
- **Desse modo, devemos ter mais cuidado a respeito de como vamos usar as IA's para nos ajudarmos.**

A **sepse** (ou infecção generalizada) é uma resposta extrema do corpo a uma infecção e é uma emergência médica com alto índice de mortalidade. Cada hora de atraso no tratamento com antibióticos diminui drasticamente as chances de sobrevivência do paciente.



A **sepse** (ou infecção generalizada) é uma resposta extrema do corpo a uma infecção e é uma emergência médica com alto índice de mortalidade. Cada hora de atraso no tratamento com antibióticos diminui drasticamente as chances de sobrevivência do paciente.

O objetivo do hospital é criar um sistema que alerte os médicos horas antes de o paciente apresentar sintomas graves, analisando o histórico do paciente, sinais vitais em tempo real e resultados de exames laboratoriais.

A **sepse** (ou infecção generalizada) é uma resposta extrema do corpo a uma infecção e é uma emergência médica com alto índice de mortalidade. Cada hora de atraso no tratamento com antibióticos diminui drasticamente as chances de sobrevivência do paciente.

O objetivo do hospital é criar um sistema que alerte os médicos horas antes de o paciente apresentar sintomas graves, analisando o histórico do paciente, sinais vitais em tempo real e resultados de exames laboratoriais.

Na sua opinião, quais são as coisas que NÃO podemos fazer usando LLM's nesse problema?



Jogar todos os dados dos pacientes

Confiar 100% em todas as respostas da LLM, sem revisar

Coisas que NÃO devem ser feitas usando LLM's

Do menos ao mais difícil de se implementar.

Fazer apenas "Comandos burros" para a máquina

Jogar todos os dados dos pacientes

Confiar 100% em todas as respostas da LLM, sem revisar

Modelos de linguagem com todas as permissões possíveis

Fazer apenas "Comandos burros" para a máquina

Jogar todos os dados dos pacientes

Problemas

- Os dados do problema não estão anonimizados, violação direta da LGPD (Lei Geral de Proteção dos Dados).
- Os registros das conversas nas LLMs podem ser expostos a hackers, levando a vazamentos dos dados.
- As conversas podem ser usadas para treinar e melhorar versões futuras do modelo, fazendo com que sejam lembradas intrinsecamente pela IA.

Jogar todos os dados dos pacientes

Confiar 100% em todas as respostas da LLM, sem revisar

Modelos de linguagem com todas as permissões possíveis

Fazer apenas "Comandos burros" para a máquina

Jogar todos os dados dos pacientes

Confiar 100% em todas as respostas da LLM, sem revisar

Problemas

- "O código gerado por IA, mesmo quando funcional, não é necessariamente seguro." [1]
- Robô tende a alucinar e a criar respostas erradas conforme o código evolui, inventando funções, parâmetros ou bibliotecas que simplesmente não existem.

Jogar todos os dados dos pacientes

Confiar 100% em todas as respostas da LLM, sem revisar

Modelos de linguagem com todas as permissões possíveis

Problemas

- Podem acabar cometendo erros sem a supervisão do ser humano, como apagar os dados.
- Suscetíveis a injeção de prompt e quebras de controle de acesso, idealizadas justamente para a segurança dos dados.

Exemplos de uso ruim

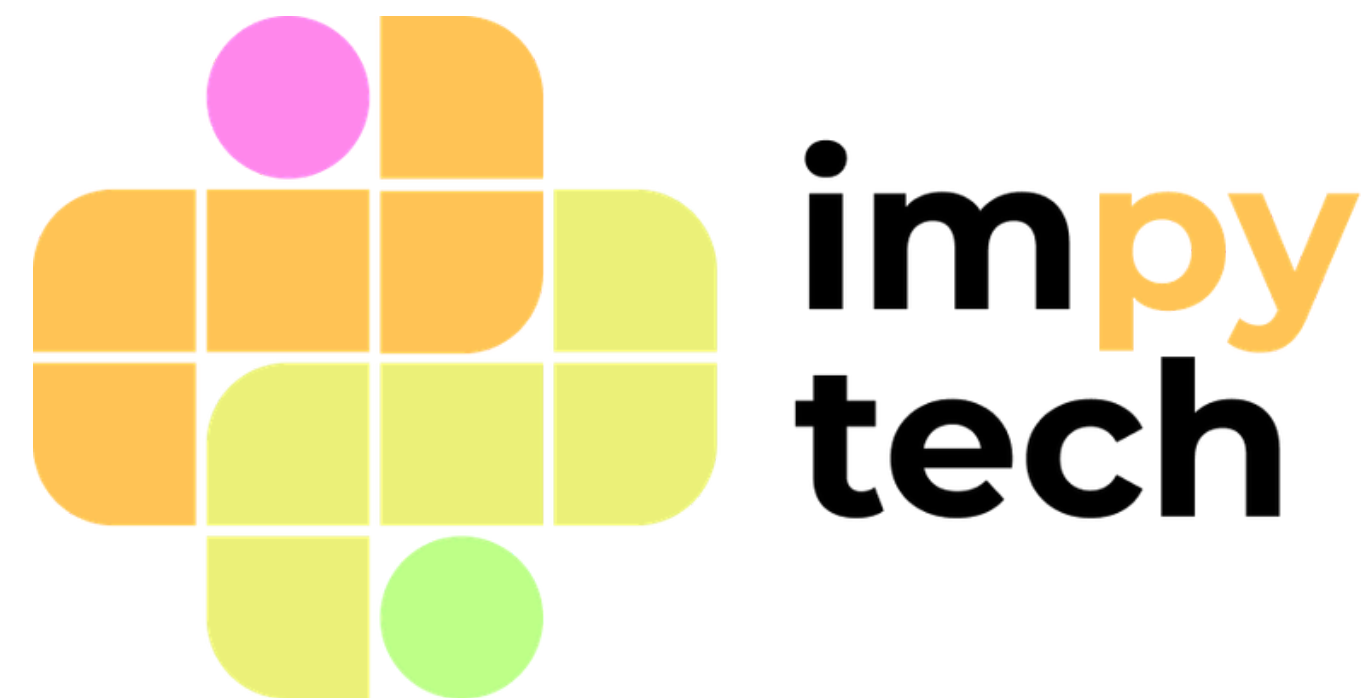


Figura 9.3: Prompt Injection sendo utilizado em processos judiciais em São Paulo

- Como se proteger:
 - Anonimize os dados: Se precisar que a IA analise um texto ou código, substitua nomes reais, CPFs, valores financeiros e dados da empresa por "Cliente A", "[CPF FALSO]" ou "Empresa X".
 - Use modelos locais: Se tiver condições, utilize modelos locais no seu computador como o Ollama, nos quais os dados não são compartilhados com um servidor externo.
 - Use versões corporativas: Empresas devem optar por versões Enterprise dessas ferramentas, que possuem contratos estritos garantindo que os dados não serão usados para treinamento e oferecem maior segurança no armazenamento.



Tema 2



Jogar todos os dados dos pacientes

Confiar 100% em todas as respostas da LLM, sem revisar

Modelos de linguagem com todas as permissões possíveis

Fazer apenas "Comandos burros" para a máquina

- Quando somos instruídos a resolver uma determinada tarefa, as explicações claras sobre como a tarefa deve ser feita geralmente mais nos ajudam do que atrapalham.

- Quando somos instruídos a resolver uma determinada tarefa, as explicações claras sobre como a tarefa deve ser feita geralmente mais nos ajudam do que atrapalham.
- A engenharia de prompt é o equivalente de esclarecermos as nossas tarefas para os robôs, com a prática de criarmos instruções claras e detalhadas aos nossos modelos de IA generativa, como o ChatGPT.

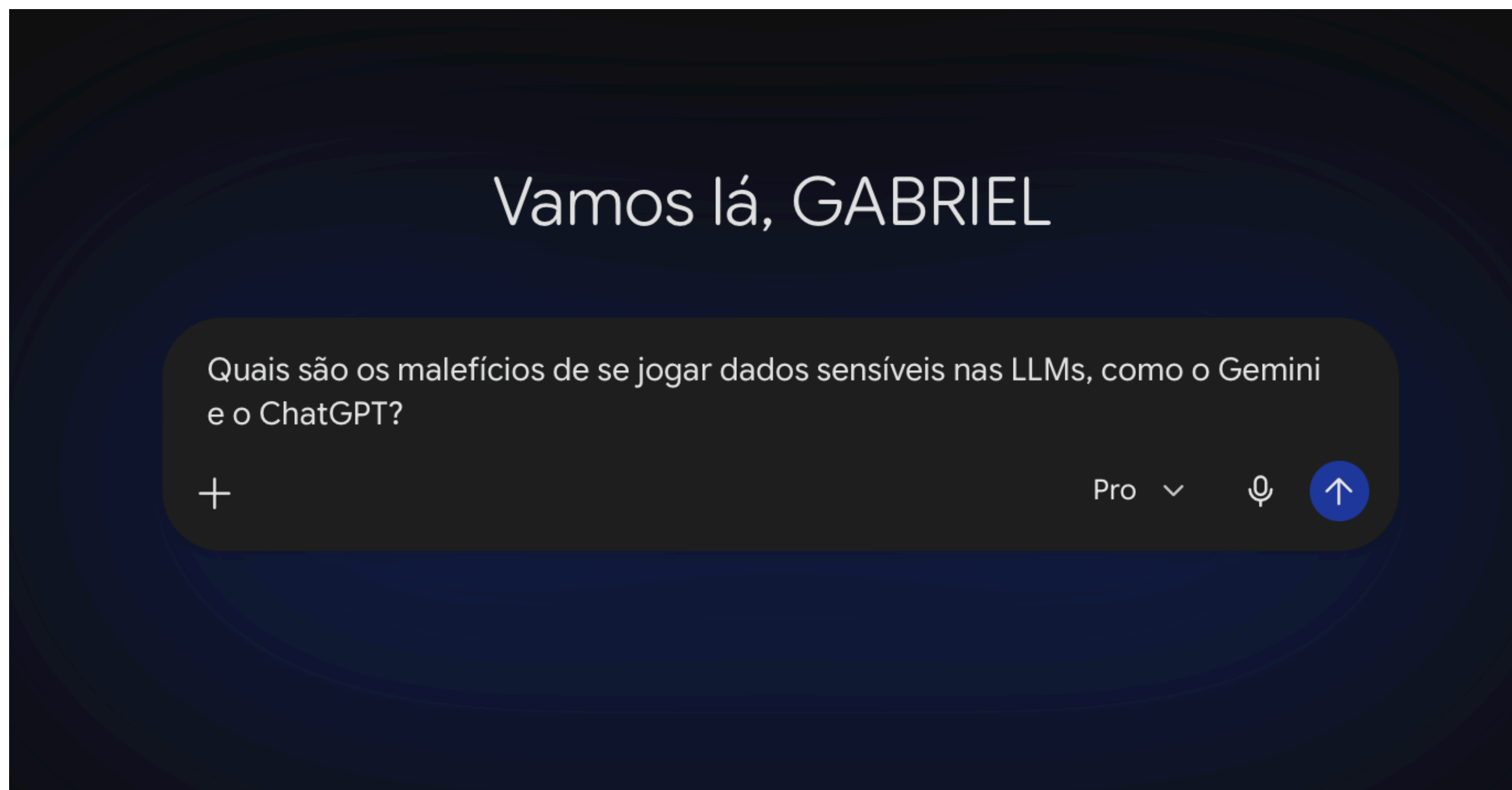


Figura 9.4: Exemplo de um prompt sendo gerado no Google Gemini.

- Existem cinco pilares fundamentais que compõem um bom prompt:
 - **Persona:** Defina o papel da IA para resolver o problema.

- Existem cinco pilares fundamentais que compõem um bom prompt:
 - **Persona:** Defina o papel da IA para resolver o problema.
 - **Contexto:** Se necessário, forneça o cenário de fundo, histórico e o objetivo principal do seu pedido. Isso facilita a IA entender o que você quer que ela faça.

- Existem cinco pilares fundamentais que compõem um bom prompt:
 - **Persona:** Defina o papel da IA para resolver o problema.
 - **Contexto:** Se necessário, forneça o cenário de fundo, histórico e o objetivo principal do seu pedido. Isso facilita a IA entender o que você quer que ela faça.
 - **Tarefa:** Seja direto e use verbos de ação para descrever exatamente o que precisa ser feito (ex: "Resuma", "Crie", "Traduza").

- Existem cinco pilares fundamentais que compõem um bom prompt:
 - **Persona:** Defina o papel da IA para resolver o problema.
 - **Contexto:** Se necessário, forneça o cenário de fundo, histórico e o objetivo principal do seu pedido. Isso facilita a IA entender o que você quer que ela faça.
 - **Tarefa:** Seja direto e use verbos de ação para descrever exatamente o que precisa ser feito (ex: "Resuma", "Crie", "Traduza").
 - **Formato:** Caso não seja óbvio, especifique a estrutura da resposta que a IA vai retornar para você.

- Existem cinco pilares fundamentais que compõem um bom prompt:
 - **Persona:** Defina o papel da IA para resolver o problema.
 - **Contexto:** Se necessário, forneça o cenário de fundo, histórico e o objetivo principal do seu pedido. Isso facilita a IA entender o que você quer que ela faça.
 - **Tarefa:** Seja direto e use verbos de ação para descrever exatamente o que precisa ser feito (ex: "Resuma", "Crie", "Traduza").
 - **Formato:** Caso não seja óbvio, especifique a estrutura da resposta que a IA vai retornar para você.
 - **Iteração:** Peça ajustes refinados, complemente as informações ou peça para a IA reescrever até atingir o resultado ideal. Provavelmente o primeiro resultado não será o ideal.

- **Persona:** Aja como um Especialista Sênior em Segurança da Informação e Privacidade de Dados.
- **Contexto:** Com a rápida adoção de LLMs (Grandes Modelos de Linguagem) como o Gemini e o ChatGPT no ambiente corporativo e pessoal, muitos usuários estão inserindo dados confidenciais, códigos-fonte proprietários e Informações de Identificação Pessoal (PII) nessas ferramentas para resumir textos ou gerar análises, sem entender como esses dados são armazenados, usados para treinamento ou os riscos de violação de privacidade (como a LGPD).
- **Tarefa:** Detalhe os principais malefícios e riscos de segurança associados à inserção de dados sensíveis nessas plataformas. Explique o que pode acontecer com essas informações e quais são as consequências para empresas e usuários comuns.
- **Formato:** Estruture sua resposta da seguinte forma:
 - 0. Uma breve introdução do cenário;
 - 1. Uma lista com *bullet points* detalhando os 4 principais riscos;
 - 2. Uma tabela simples comparando "Exemplo de Dado Sensível Inserido" vs "Possível Consequência".
Mantenha um tom profissional, didático e de alerta.
- **Iteração:** Ao final da sua resposta, pergunte-me se o nível de detalhamento técnico está adequado ou se eu gostaria que você criasse um pequeno manual de "Boas Práticas" para mitigar esses riscos.

Figura 9.5: Estrutura melhorada do exemplo de prompt.

Jogar todos os dados dos pacientes

Confiar 100% em todas as respostas da LLM, sem revisar

Modelos de linguagem com todas as permissões possíveis

Fazer apenas "Comandos burros" para a máquina

Jogar todos os dados dos pacientes

Confiar 100% em todas as respostas da LLM, sem revisar

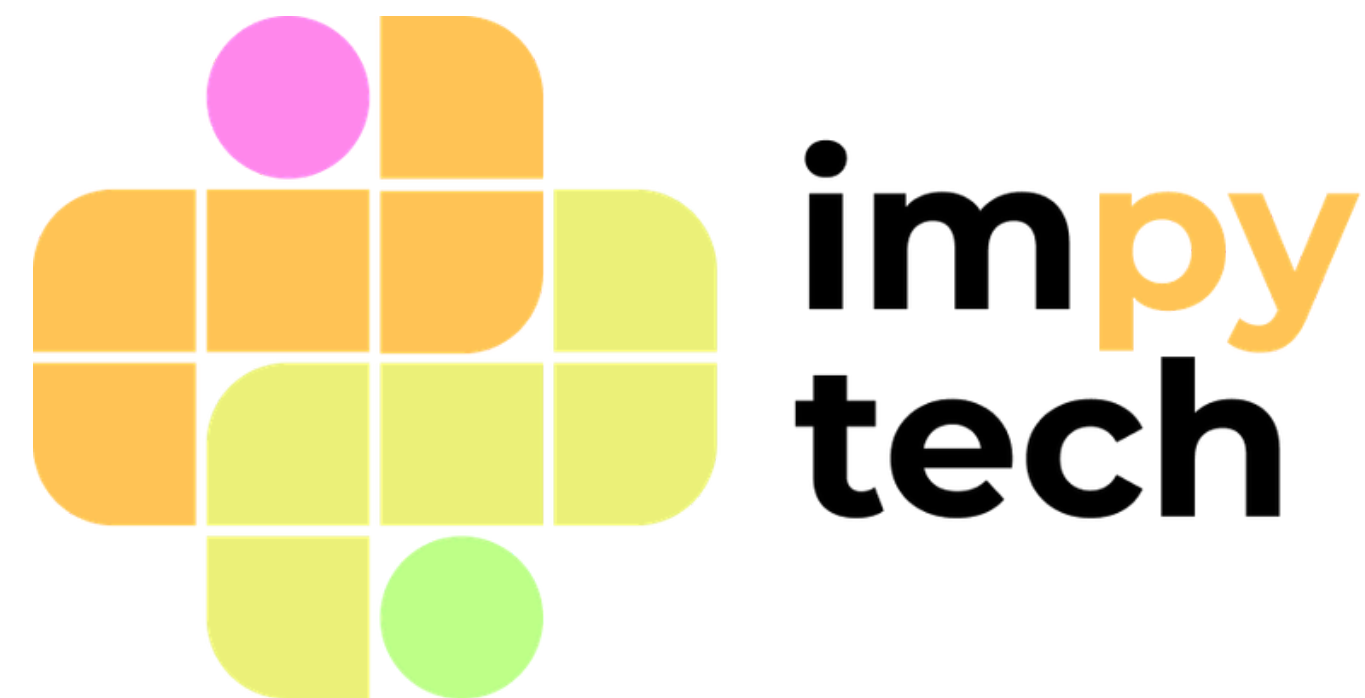
Modelos de linguagem com todas as permissões possíveis

Fazer apenas "Comandos burros" para a máquina

Devemos ter **responsabilidade** ao usar LLMs



Tema 3



- Ao ter o controle, você garante que todos os passos onde foram utilizados alguma forma de inteligência artificial são legíveis, compreendido por você e que provavelmente vão funcionar, diminuindo a capacidade de acontecerem erros no código gerado.

- Ao ter o controle, você garante que todos os passos onde foram utilizados alguma forma de inteligência artificial são legíveis, compreendido por você e que provavelmente vão funcionar, diminuindo a capacidade de acontecerem erros no código gerado.
- A nível de aprendizado, ainda é difícil de conseguirmos obter "novos conhecimentos", pois depende que o computador consiga abstrair coisas nunca raciocinadas antes, o que provavelmente não é fácil.

- Ao ter o controle, você garante que todos os passos onde foram utilizados alguma forma de inteligência artificial são legíveis, compreendido por você e que provavelmente vão funcionar, diminuindo a capacidade de acontecerem erros no código gerado.
- A nível de aprendizado, ainda é difícil de conseguirmos obter "novos conhecimentos", pois depende que o computador consiga abstrair coisas nunca raciocinadas antes, o que provavelmente não é fácil.
- Com isso, ainda cabe a nós, seres humanos, tomarmos as decisões necessárias para que as descobertas junto com as IAs sejam possíveis.

arXiv:2204.01467v1 [cs.CY] 4 Apr 2022

On scientific understanding with artificial intelligence

Mario Krenn,^{1,2,3,4,*} Robert Pollice,^{2,3} Si Yue Guo,² Matteo Aldeghi,^{2,3,4} Alba Cervera-Lierta,^{2,3} Pascal Friederich,^{2,3,5} Gabriel dos Passos Gomes,^{2,3} Florian Häse,^{2,3,4,6} Adrian Jinich,⁷ AkshatKumar Nigam,^{2,3} Zhenpeng Yao,^{2,8,9,10} and Alán Aspuru-Guzik^{2,3,4,11,†}

¹Max Planck Institute for the Science of Light (MPL), Erlangen, Germany.
²Chemical Physics Theory Group, Department of Chemistry, University of Toronto, Canada.
³Department of Computer Science, University of Toronto, Canada.
⁴Vector Institute for Artificial Intelligence, Toronto, Canada.
⁵Institute of Nanotechnology, Karlsruhe Institute of Technology, Eggenstein-Leopoldshafen, Germany.
⁶Department of Chemistry and Chemical Biology, Harvard University, Cambridge, USA.
⁷Division of Infectious Diseases, Weill Department of Medicine, Weill-Cornell Medical College, New York, USA.
⁸Center of Hydrogen Science, Shanghai Jiao Tong University, 800 Dongchuan Road, Shanghai 200240, China.
⁹The State Key Laboratory of Metal Matrix Composites, School of Materials Science and Engineering, Shanghai Jiao Tong University, 800 Dongchuan Road, Shanghai 200240, China.
¹⁰Innovation Center for Future Materials, Zhangjiang Institute for Advanced Study, Shanghai Jiao Tong University, 429 Zhangheng Road, Shanghai 201203, China.
¹¹Canadian Institute for Advanced Research (CIFAR) Lebovic Fellow, Toronto, Canada.

Imagine an oracle that correctly predicts the outcome of every particle physics experiment, the products of every chemical reaction, or the function of every protein. Such an oracle would revolutionize science and technology as we know them. However, as scientists, we would not be satisfied with the oracle itself. We want more. We want to comprehend how the oracle conceived these predictions. This feat, denoted as scientific understanding, has frequently been recognized as the essential aim of science. Now, the ever-growing power of computers and artificial intelligence poses one ultimate question: How can advanced artificial systems contribute to scientific understanding or achieve it autonomously?

We are convinced that this is not a mere technical question but lies at the core of science. Therefore, here we set out to answer where we are and where we can go from here. We first seek advice from the philosophy of science to *understand scientific understanding*. Then we review the current state of the art, both from literature and by collecting dozens of anecdotes from scientists about how they acquired new conceptual understanding with the help of computers. Those combined insights help us to define three dimensions of android-assisted scientific understanding: The android as a I) computational microscope, II) resource of inspiration and the ultimate, not yet existent III) agent of understanding. For each dimension, we explain new avenues to push beyond the status quo and unleash the full power of artificial intelligence's contribution to the central aim of science. We hope our perspective inspires and focuses research towards androids that get new scientific understanding and ultimately bring us closer to true artificial scientists.

I. INTRODUCTION

Artificial Intelligence (A.I.) has recently been called a “new tool in the box for scientists” [1] and that “machine learning with artificial networks is revolutionizing science” [2]. Additionally, it has been conjectured “that machines could have a significantly more creative role in future research.” [3]. For instance, it has even been postulated that “[t]he new goal of theoretical chemistry should be that of providing access to a chemical ‘oracle’: an A.I. environment which can help humans solve problems, associated with the fundamental chemical questions of the fourth industrial revolution [...], in a way such that the human cannot distinguish between this and communicating with a human expert” [4].

However, this excitement has not been shared among all scientists. Specifically, it has been questioned whether advanced computational approaches can go beyond *numerics* [5–9] and contribute fundamentally to one of the essential aims of science, that is, gaining of new scientific understanding [10–12].

In this work, we address how artificial systems can contribute to scientific understanding – specifically, what is the state-of-the-art and how we can push further. Besides a thorough literature review, we surveyed dozens of scientists at the interface of biology, chemistry or physics on the one hand, and artificial intelligence and advanced computational methods. These personal narratives focus on the concrete discovery process of ideas and are a vital augmentation to the scientific literature. We put the literature and personal accounts in the context of a philosophi-

* mario.krenn@mpl.mpg.de
† alan@aspuru.com

Figura 9.6: Artigo "On Scientific Understanding with Artificial Inteligence", que debate acerca da compreensão científica.



Ollama

Ollama is the easiest way to automate your work using open models, while keeping your data safe.

 ollama

Referências bibliográficas:

- <https://www2.ufjf.br/cideng/2026/05/15/geracao-de-codigo-por-modelos-de-linguagem-llms-potencialidades-riscos-e-responsabilidades-eticas/>
- <https://www.hostgator.com.br/blog/engenharia-de-prompt/>
- Introduction to statistical learning: Chapters 1 and 2
- Pense em Python (1): Capítulos 15, 16, 17 e 18

Leitura Adicional

- KRENN, Mario et al. On scientific understanding with artificial intelligence. Nature Reviews Physics, v. 4, n. 12, p. 761–769, dez. 2022. Link 2
- Material do curso de Machine Learning do IMPATech: <https://rbribeiro.github.io/md22/>

Github



Referências bibliográficas:

- Pense em Python (1): Capítulos 15, 16, 17 e 18

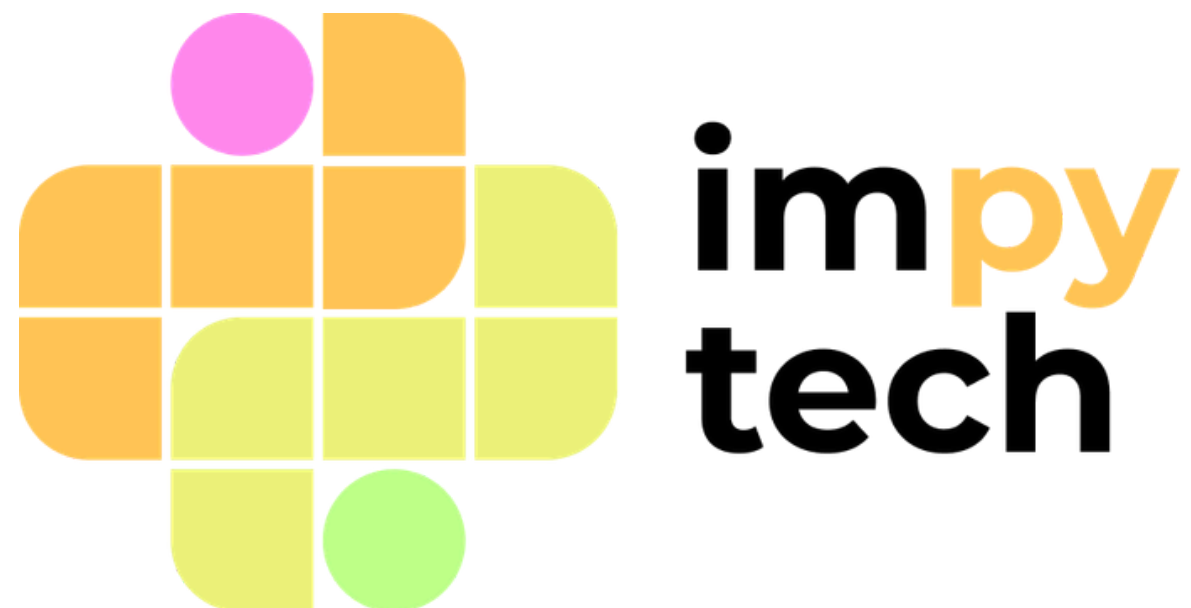
Exercícios

- Jupyter de exercícios X
- Link para exercícios



Livro 1





Obrigado!

Avalie a aula

